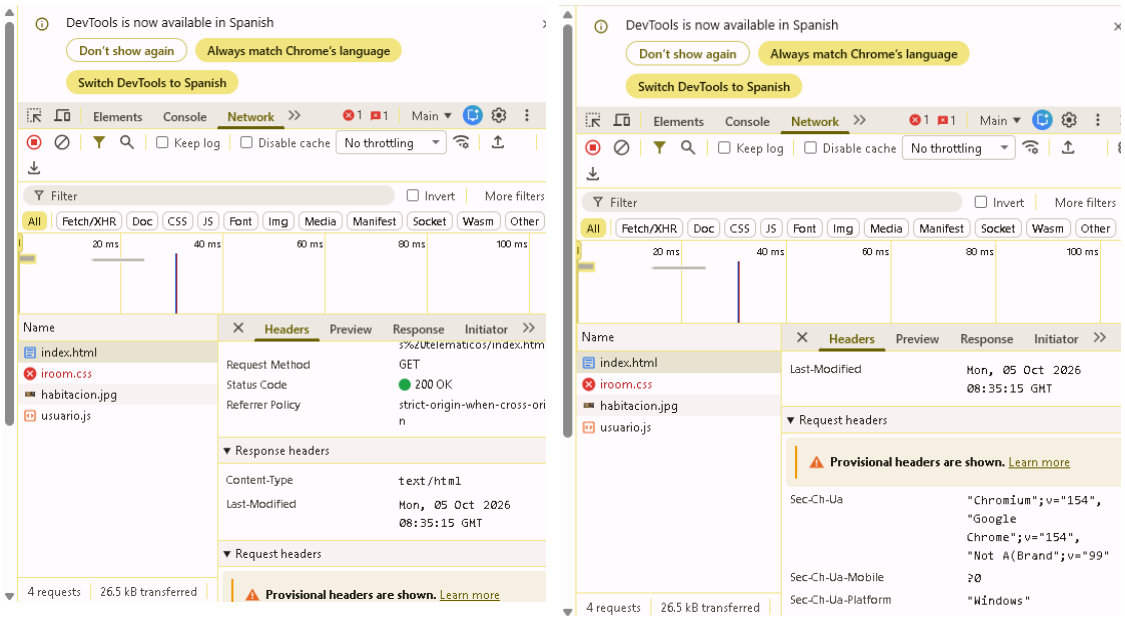
PRÁCTICA 1 : HTML/CSS

Apartado 4. Control de caché

Para comprobar cómo gestiona el navegador la carga de los archivos de la aplicación, he abierto las herramientas de desarrollo con (Ctrl + Shift + I) o (click derecho + inspeccionar) y he accedido a la pestaña Red (Network). Tras recargar la página index.html, he seleccionado la petición del documento principal(index.html) para inspeccionar sus cabeceras de solicitud (Request Headers) y cabeceras de respuesta (Response Headers).



Cache-Control no-cache: Le dice al navegador que sí puede guardar una copia del archivo en su memoria local, pero no puede usarla directamente. Cada vez que entro a la página, el navegador tiene que preguntarle primero al servidor si el archivo ha cambiado . Si el servidor responde que no hay cambios (código 304 Not Modified), el navegador reutiliza la versión guardada.

Cache-Control no-store: Es más estricto. Le prohíbe al navegador (o a cualquier servidor intermedio) guardar o almacenar cualquier copia del archivo. Cada vez que se pide la página, el navegador la descarga de cero desde el servidor. Se usa para datos privados o sensibles.

Recarga normal (F5): El navegador intenta ser eficiente. Utiliza los datos que ya tiene en caché y solo pregunta al servidor si han cambiado los archivos para descargarlos únicamente si es necesario.

Recarga forzada (Ctrl + F5 o Ctrl + Shift + R): El navegador ignora por completo todo lo que guardó en la caché local. Envía una petición limpia al servidor obligándole a devolver todos los archivos desde cero (obteniendo respuestas 200 OK).

En cuanto los archivos estáticos de nuestra web como la hoja de estilos (iroom.css), las imágenes (habitacion.jpg) o los scripts (usuario.js) no cambian continuamente a diferencia del contenido HTML. Por eso, en estos archivos conviene usar políticas de caché con tiempos de caducidad largos, evitando descargas innecesarias y haciendo que la web cargue más rápido. Sin embargo, si modificamos el CSS o el JavaScript, un navegador que tenga guardada la versión antigua en caché no mostrará los cambios a los usuarios. Para solucionar esto se utiliza el versionado de recursos (Cache Busting), que consiste en añadir un parámetro de versión a la URL del archivo en el HTML (por ejemplo, ./css/iroom.css?v=1.1). Al cambiar este valor en el enlace, el navegador detecta que es una URL "nueva" y descarga el archivo actualizado inmediatamente.

Apartado 6. Actualización mediante eventos enviados por el servidor

Se propone una arquitectura cliente-servidor basada en HTTP donde se establece un canal de comunicación unidireccional (simplex) persistente sobre TCP. El cliente inicia la sesión con una única petición GET y el servidor mantiene el socket abierto. Esto permite hacer un push asíncrono de los datos de los sensores hacia el frontend en tiempo real en cuanto hay una nueva lectura, reduciendo el overhead de cabeceras HTTP y la latencia en comparación con técnicas clásicas como el polling periódico.

En cuanto al formato de los mensajes text/evento-stream , en la capa de aplicación, el servidor backend debe negociar el flujo respondiendo con la cabecera HTTP Content-Type: text/event-stream. El payload transmitido es texto plano codificado en UTF-8 y estructurado en bloques (frames). Cada mensaje de los sensores debe separarse por dos saltos de línea y usar un tipado estándar. Por ejemplo:

```
event: update_sensor
data: {"id_sensor": "temp_01", "valor": 24.5, "unidad": "C"}
id: 1045
```

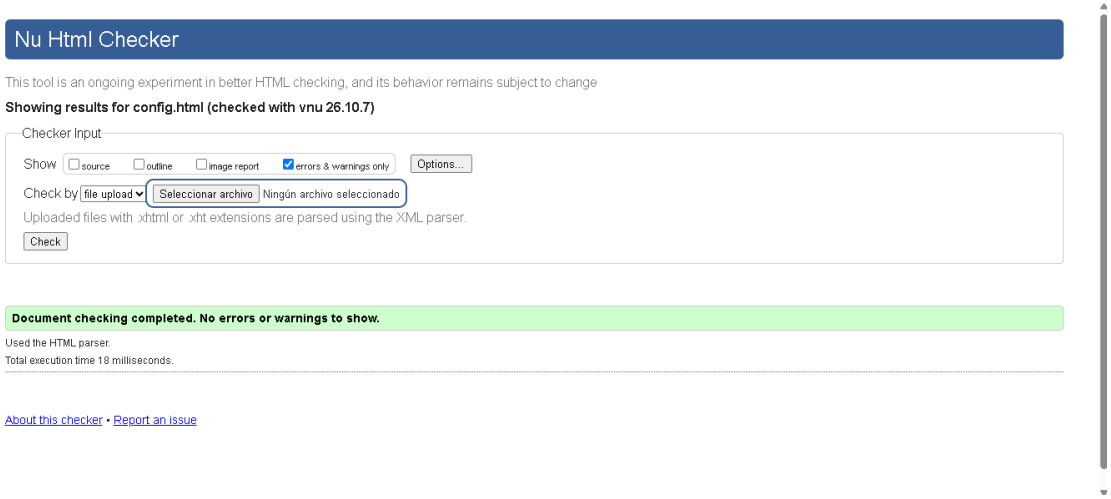
La conexión mediante EventSource , en la parte del cliente (JavaScript), instanciamos el objeto nativo EventSource apuntando al endpoint del servidor que emite el stream. Esta abstracción resuelve a bajo nivel el mantenimiento de la sesión y los intentos de reconexión automática en caso de caída del enlace de red:

```
// Instanciamos el socket de escucha hacia el backend
const flujoSensores = new EventSource('http://iroom.uah.es/api/stream-sensores');
```

Para el procedimiento de actualización de la interfaz , se emplea un paradigma orientado a eventos de manera asíncrona. Se suscriben callbacks al objeto instanciado. Cuando llega un nuevo datagrama por el socket, la interrupción dispara la función, se parsea el JSON recibido y se ataca el DOM directamente para refrescar el nodo específico del sensor, evitando así recargar la página entera y optimizando el renderizado:

```
flujoSensores.addEventListener('update_sensor', (evento) => {
```

Apartado 9. Validación y calidad del código



Esta es la captura de lo que aparece al comprobar el código , con todos los archivos

La herramienta utilizada es un comprobador online : validator.w3.org

Los problemas identificados al comprobar los archivos han sido separaciones , es decir , que no se cumplia el espacio recomendado por la sintaxis , la correccion ha sido muy sencilla ha sido , separar para que se cumpla el espacio que pide y asi no tener el error.

Como nota final los problemas que hemos tenido durante la realización de la práctica han sido principalmente , que no nos dimos cuneta al principio de la practica de instalar git , y por tanto no estabamos subiendo bien los archivos a github , por lo que cuando ya teniamos la practica hecha nos dimos cuenta y lo hicimos , y esto nos dio problemas.